OpenClaw Memory System: Architecture & Technical Deep Dive

nox-mem v3.7 — March 2026, §5 empirical evaluation added May 2026 (Wave A G5 V3)

Author: Luiz Antonio Busnello (Toto) **Platform:** OpenClaw Autonomous Agent Platform **Infrastructure:** Hostinger KVM4, Tailscale VPN, Debian Linux

---

Abstract

We introduce **nox-mem**, a persistent memory system for autonomous LLM agents organized around a single design principle: **pain-weighted hybrid memory with shadow discipline — yours by design**. Every retrieval and retention decision is governed by a `salience = recency × pain × importance` formula (where *pain* is an operator-assigned severity in [0.1, 1.0], persisted on every chunk), and every ranking change is gated by a mandatory shadow-mode telemetry phase before activation in production. Compared to existing memory systems (mem0, Letta, Zep, EverOS, LightRAG, and MeMo), nox-mem offers several advantages: **(a)** *live writeback with sub-second indexing* (inotifywait-driven, no batch retrain or daily reindex required), **(b)** *typed temporal decay* (per-`chunk_type` retention windows with never-decay for feedback/person — see §2 and §8.2), **(c)** *full provenance to chunk and source* (every retrieval result carries `chunk_id` + `source_file`, every destructive op is wrapped in `withOpAudit()` with a VACUUM INTO pre-snapshot), **(d)** *first-class self-evolution* through a `crystallize/reflect/consolidate` triad that promotes high-hit pending items to durable lessons and synthesizes cross-session insights nightly (§3.4), and **(e)** *zero vendor lock-in*: a single SQLite file, MIT-licensed, with provider-agnostic embeddings (Gemini default, swappable). Deployed in production since March 14, 2026, the system manages 62.9k+ memory chunks with ~99.97% vector coverage, ~402 knowledge graph entities with ~544 relations, and serves 6 specialized AI agents with isolated yet interconnectable memory spaces. On the entity-flavored golden set (n=100), the full ablation stack reaches **nDCG@10 = 0.6237** (+78.8% over the G3 pre-Wave-A baseline; §5), and the Q4 cross-system comparison (§6) provides pre-registered, head-to-head numbers against five competing memory systems on a shared corpus and harness.

---

1. Introduction

1.1 Problem Statement

Large Language Model (LLM) agents operating in production environments face a fundamental limitation: context window ephemerality. When a conversation

ends or context is compacted, the agent loses accumulated knowledge, decisions, and operational state. For multi-agent systems where specialized agents collaborate on complex tasks, this problem compounds — agents cannot learn from each other's experiences, cannot recall past decisions, and cannot build institutional knowledge over time.

1.2 Design Goals

nox-mem was designed with four core objectives:

1. **Persistent Memory**: Survive context window resets, session boundaries, and agent restarts
2. **Intelligent Retrieval**: Return semantically relevant results, not just keyword matches
3. **Cross-Agent Intelligence**: Enable knowledge sharing across isolated agent workspaces
4. **Operational Autonomy**: Self-maintain through automated consolidation, pruning, and indexing

1.3 Scope

The system operates within the OpenClaw platform, serving 6 AI agents (Nox, Atlas, Boris, Cipher, Forge, Lex) on a single VPS with 4 vCPUs and 8GB RAM. Each agent has a distinct role and memory profile. The workspace (shared memory) and individual agent databases form a federated memory architecture.

1.4 Related Memory Systems and the Six Gaps

The published memory-for-LLM-agents literature spans roughly three families: (i) *vector-store wrappers with metadata layers* — **mem0** [1], **Letta** [2]; (ii) *temporal- and provenance-aware memory services* — **Zep** [3], **memanto**; (iii) *KG-augmented and graph-fused retrieval* — **LightRAG** [4] (HKU, EMNLP 2025), **HippoRAG2** [5]; and (iv) *parametric-memory paradigms*, most recently

[1] mem0ai/mem0 — open-source memory layer for LLM agents (PostgreSQL + Qdrant backend, OpenAI embeddings by default). github.com/mem0ai/mem0. Used in §1.4, §6.3, Table 2.

[2] Letta (formerly MemGPT) — agent-loop memory architecture with archival/recall memory separation. github.com/letta-ai/letta. Used in §1.4, §6.3, Table 2.

[3] Zep — temporal knowledge-graph memory service with summarization. github.com/getzep/zep. OpenAI embedding is the hardcoded default in the OSS distribution. Used in §1.4, §6.3, Table 2.

[4] Guo et al., *LightRAG: Simple and Fast Retrieval-Augmented Generation*, EMNLP 2025 (HKU). github.com/HKUDS/LightRAG (~35k stars, MIT). Cited in §1.4 as a KG-augmented baseline; §3.4 references its LLM-summarized incremental KG-merge pattern as a forward-looking optimization (LightRAG-style summarization parking-lotted until KG density >=10× current; see `docs/COMPETITIVE-ANALYSIS-2026-05-19.md`).

[5] HippoRAG2 — graph-augmented retrieval with Personalized PageRank over an entity-relation graph. Cited as a graph-baseline peer in §1.4 and §6.

MeMo [6] (which folds reflections into model weights via continued pretraining — the design opposite of ours). **EverMind-AI/EverOS** [7] occupies a distinct slot: it is the only memory OS in this space that publishes its own benchmark dataset (EverMemBench) and reports threshold numbers, raising the bar for honest cross-system comparison (§6).

Across these systems, six recurring gaps appear in the design space. Each gap motivates a concrete subsystem of nox-mem. Table 1 summarizes who covers what:

Table 1 — Comparison of desirable memory-system properties (Six Gaps).

#	Gap	nox-mem	mem0	Letta	Zep	EverOS	LightRAG	MeMo
1	Static injection (memory frozen at session start)	Y live write-back (inotifywait <1s)	~ partial	Y	Y	Y	N batch	N retrain

[6] arXiv 2605.15156v2, *MeMo: Towards Language Models with Associative Memory Mechanisms* (parametric reflections folded into model weights). Cited as the design opposite of nox-mem's externalized, inspectable memory paradigm. §1.4, abstract.

[7] EverMind-AI / EverOS — github.com/EverMind-AI (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. The only memory-OS peer in our taxonomy that publishes its own benchmark; §3.4 is the direct narrative counter to EvoAgent framing. Honest-comparison action item: run nox-mem on EverMemBench (queued as F4 in §7.2).

#	Gap	nox-mem	mem0	Letta	Zep	EverOS	LightRAG	MeMo
2	No temporal decay (every chunk weighed the same forever)	Y salience x recency, typed retention (§8.2)	N	N	Y	~ partial	N	N
3	No provenance (cannot trace a result back to a source)	Y `chunk_id` + `source_file` (§4)	~ partial	Y	Y	Y	Y	N baked in weights
4	Flat memory (no hierarchy / sectioning / typed structure)	Y KG + entity files + `section_boost` (§3.1, §8)	N	N	Y	Y hyper	Y dual-level	N

#	Gap	nox-mem	mem0	Letta	Zep	EverOS	LightRAG	MeMo
5	No write-back (cannot promote findings into durable memory)	Y `crystallize` + `reflect` + `consolidate` (§3.4)	~ partial	Y	Y	Y EvoAgent	N	N
6	Indexing delay (changes<1s, invisible until nightly batch)	Y inotify-wait idempotent re-ingest (§3.1)	~	Y	Y	~	~ batch	N retrain

Why each gap matters, and how nox-mem closes it:

1. **Static injection.** A memory system that only reads at the start of a session cannot observe what the agent does *during* the session. nox-mem's watcher [8] re-ingests every modified `.md` / `.json` file within ~1 second of the write, with idempotent chunk replacement (§3.1).
2. **No temporal decay.** Treating a daily note from 91 days ago the same as a crystallized decision floods retrieval with stale noise. nox-mem assigns each `chunk_type` a `retention_days` window (V8 schema column; `feedback`/`person` = never-decay, `lesson` = 180d, `decision`/`project` = 365d, `daily` = 90d) and folds it into the salience formula (§8.2).
3. **No provenance.** Parametric and opaque-vector memories cannot answer "*why* did you return this?". Every nox-mem result carries the originating `chunk_id` and `source_file`, every destructive operation goes through `withOpAudit()` with a `VACUUM INTO` pre-snapshot

[8] `nox-mem-watcher` systemd service running `inotifywait` on `memory/` directories. See `docs/ARCHITECTURE.md` and §3.1 of this paper.

to `/var/backups/nox-mem/pre-op/`, and the `ops_audit` table is append-only.

4. **Flat memory.** Without structure, hybrid search collapses to "more recent or more frequent wins". nox-mem layers (a) FTS5 over chunk text, (b) typed retention, (c) an LLM-extracted knowledge graph (§8), and (d) an entity-file format (`frontmatter` / `compiled` / `timeline` sections) with section-aware `section_boost` — the dominant driver of the +78.8% gain in §5.
5. **No writeback.** A system that can only *be read* cannot grow. nox-mem's self-evolution triad — `crystallize`, `reflect`, `consolidate` — is described in §3.4 and is the most direct counter to EverOS's EvoAgentBench framing [9].
6. **Indexing delay.** Daily-batch reindex makes "new memory" a 24h-resolution operation. inotifywait makes it sub-second (§3.1), and `--dry-run` mode plus `withOpAudit()` snapshots make destructive ops (reindex, consolidate, compact, crystallize, kg-prune) reversible.

The single design principle that ties these closures together is **pain weighting under shadow discipline**: every chunk carries an explicit `pain` severity, every ranking change rolls out first in `NOX_SALIENCE_MODE=shadow` with /api/health telemetry [10], and only graduates to `active` after operator review.

---

2. System Architecture

2.1 Infrastructure Overview

The system runs on a Hostinger KVM4 VPS accessible via Tailscale VPN (IP: 100.87.8.44). Five systemd-managed services provide the runtime environment:

Service	Port	Type	Function
openclaw-gateway	18789	WebSocket	Agent communication gateway
nox-mem-watcher	—	inotifywait	Filesystem event monitor
nox-mem-api	18800	HTTP/JSON	Dashboard data API
ollama	11434	HTTP	Local LLM inference (llama3.2:3b)
tailscaled	—	WireGuard	VPN mesh connectivity

[9] EverMind-AI / EverOS — github.com/EverMind-AI (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. The only memory-OS peer in our taxonomy that publishes its own benchmark; §3.4 is the direct narrative counter to EvoAgent framing. Honest-comparison action item: run nox-mem on EverMemBench (queued as F4 in §7.2).

[10] `NOX_SALIENCE_MODE` environment variable controls the three-state gate (`shadow` | `active` | `off`). Default is `shadow`. Telemetry exposed at `/api/health.salience`. See §3.4.3, `staged-1.7a/edits/salience.ts`, and `docs/CONFIGURATION.md`.

2.2 Database Schema

The primary storage is SQLite 3 with WAL (Write-Ahead Logging) mode for concurrent access. The schema (version 3) contains:

Core Tables:

- `chunks` — Memory fragments with full-text indexing
  - `id` (INTEGER PK), `source_file` (TEXT), `chunk_text` (TEXT), `chunk_type` (TEXT)
  - `source_date` (TEXT), `is_consolidated` (INTEGER), `memory_type` (TEXT)
  - `created_at`, `updated_at` (TEXT, ISO 8601)
  - `metadata` (TEXT, JSON)
- `chunks_fts` — FTS5 virtual table with porter unicode61 tokenizer
  - Content-sync triggers (INSERT, UPDATE, DELETE) maintain index consistency
  - BM25 ranking with configurable column weights (1.0, 0.5, 0.5)
- `consolidated_files` — Processing state tracker
  - `source_file` (TEXT PK), `status` (INTEGER: 0=pending, 1=done, -1=failed)
- `meta` — Key-value configuration store (schema_version, cursors, metrics)

Knowledge Graph Tables:

- `kg_entities` — Named entities with type classification and mention counting
  - UNIQUE constraint on (name, entity_type)
  - TTL tracking via `first_seen`, `last_seen`
- `kg_relations` — Typed relationships between entities
  - Confidence scoring (0.0-1.0) with temporal decay
  - TTL via `expires_at` (90-day default), `last_confirmed`
  - Evidence linking via `evidence_chunk_id`
- `decision_versions` — Architectural decision version history
  - Supersession chain via `is_current` flag and `superseded_at` timestamp

Vector Tables (sqlite-vec):

- `vec_chunks` — Virtual table storing float32 embeddings (3072 dimensions)
- `vec_chunk_map` — Rowid-to-chunk_id mapping (sqlite-vec requires rowid-based access)
- `dedup_log` — Suppressed duplicate tracking for audit

2.3 Chunk Type Taxonomy

Memory chunks are classified into 10 types based on source file path patterns:

Type	Source Pattern	Current Count	Purpose
team	`shared/`	499	Shared team knowledge, cross-agent docs
daily	`memory/YYYY-MM-DD`	161	Daily operational notes
other	(default)	126	Unclassified content
decision	`memory/decisions.md`	34	Architectural and strategic decisions
lesson	`memory/lessons.md`	21	Errors, corrections, learnings
project	`memory/projects.md`	11	Active project tracking
pending	`memory/pending.md`	8	Incomplete tasks and blockers
feedback	`memory/feedback/`	6	User and system feedback
person	`memory/people.md`	6	People profiles and contacts
digest	`memory/digests/`	2	Weekly summary reports

2.4 Multi-Agent Memory Architecture

Each of the 6 agents operates with an isolated database at `/root/.openclaw/agents/{name}/tools/nox-mem/nox-mem.db`. The `OPENCLAW_WORKSPACE` environment variable controls path resolution across all modules, enabling the same nox-mem binary to operate on different databases depending on the calling context.

Agent Memory Distribution (as of March 23, 2026):

Agent	Role	Chunks	DB Size	Dominant Type
Nox	Chief of Staff	185	268 KB	daily (91)
Boris	Head of Communications	148	268 KB	team (50)
Forge	Code Reviewer	182	292 KB	daily (135)

Agent	Role	Chunks	DB Size	Dominant Type
Atlas	Research	30	128 KB	other (17)
Cipher	Security	31	132 KB	other (12)
Lex	Legal/Compliance	31	132 KB	other (12)
Workspace	**Shared**	**874**	**25.2 MB**	**team (499)**

Total system memory: 1,481 chunks across 7 databases.

---

3. Memory Pipeline

3.1 Ingestion

Files created or modified in monitored directories trigger the inotifywait-based watcher service. The watcher implements:

- **Debounce logic**: 2-second delay to batch rapid successive writes
- **File filtering**: Only `.md` and `.json` files are processed
- **Recursion prevention**: `MEMORY.md` and `SESSION-STATE.md` are excluded to avoid feedback loops
- **Heartbeat**: Touch `/tmp/nox-mem-watcher-heartbeat` on every event for liveness monitoring

Upon trigger, `ingestFile()` executes:

1. Read file content with UTF-8 sanitization (fixes common mojibake patterns for Portuguese text)
2. Detect chunk type from relative file path
3. Extract date from filename pattern (YYYY-MM-DD)
4. Split content into semantic chunks:
   - Markdown: Split on H2/H3 headers, with sub-splitting for chunks exceeding 500 words
   - JSON: Array items become individual chunks; object entries become key-value pairs
   - Small chunks (<20 words) are merged with the previous chunk
5. Delete existing chunks for the same source file (idempotent re-ingestion)
6. Insert new chunks via prepared statement transaction
7. Auto-vectorize if GEMINI_API_KEY is available (up to 20 chunks per file)

3.2 Consolidation

Nightly consolidation (23:00, 5-minute stagger across agents) processes daily notes into structured topic files:

1. **Reindex**: Scan all `.md`/`.json` files in memory directories, rebuild chunk index

2. **Extract**: Use Ollama llama3.2:3b to identify facts, decisions, lessons, and action items from daily notes
3. **Append**: Add extracted content to topic files (decisions.md, lessons.md, people.md, projects.md, pending.md)
4. **Notion Sync**: Push structured items to "Memoria & Decisoes" Notion database (best-effort, non-blocking)
5. **Git Commit**: Auto-commit memory changes with standardized message format
6. **Session Update**: Refresh SESSION-STATE.md with current statistics

3.3 Deduplication

Before insertion, chunks are checked for duplicates using a two-tier strategy:

- **Primary**: Gemini cosine similarity with 0.85 threshold (when embeddings are available)
- **Fallback**: Keyword overlap calculation with 60% threshold
- **Audit**: Suppressed duplicates are logged to `dedup_log` table with reason and preview

3.4 Self-Evolution: How nox-mem Improves Over Time

Most memory systems decouple *retrieval* from *learning*: retrieval is online, learning is either absent or requires retraining a parametric backbone (MeMo) or relying on a closed evolution loop (EverOS EvoAgentBench [11]). nox-mem instead promotes self-evolution to a first-class subsystem composed of four cooperating mechanisms, all transparent and inspectable in the live SQLite file. This subsection is the direct counter to Gap #5 (no writeback) in Table 1.

3.4.1 Crystallize loop — `pending` → `lesson` after N hits, with pain accumulation.

The `crystallize` command (exposed via the CLI, the MCP server tool, and `POST /api/crystallize` + `POST /api/crystallize/validate` on the HTTP API; see `src/api-server.ts` and `src/crystallize.ts` [12]) implements an LLM-assisted consolidation that synthesizes durable entities from recent chunks. Operationally, chunks ingested into `memory/pending.md` (`chunk_type = pending`, `retention_days = 30`) act as a working set of unresolved items. As the same pending item is referenced by retrieval (and as related daily/team chunks accumulate around it), its effective *pain*

[11] EverMind-AI / EverOS — github.com/EverMind-AI (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. The only memory-OS peer in our taxonomy that publishes its own benchmark; §3.4 is the direct narrative counter to EvoAgent framing. Honest-comparison action item: run nox-mem on EverMemBench (queued as F4 in §7.2).

[12] HTTP handlers in `src/api-server.ts` (routes `/api/crystallize`, `/api/crystallize/validate` — confirmed in `staged-1.6/edits/api-server.ts:253-273`); core logic in `src/crystallize.ts` exporting `crystallize()`, `validateProcedure()`, `listProcedures()`. Wrapped by `withOpAudit()` (`src/lib/op-audit.ts`).

signal grows — both via direct operator updates to the `pain` column (V9 schema) and via co-occurrence with high-pain neighbors. After enough hits and sufficient accumulated severity, `crystallize` uses Gemini to identify the pattern across chunks, emits a canonical entity file under `memory/entities/<type>/<slug>.md`, and effectively promotes the cluster from `pending` (30-day decay) to `lesson` (180-day decay) or `decision`/`project` (365-day decay) [13]. The promotion is wrapped in `withOpAudit()`, so the pre-state snapshot lives at `/var/backups/nox-mem/pre-op/crystallize-<ts>-<pid>-<uuid>.db` and a row lands in the append-only `ops_audit` table.

3.4.2 Pain weighting auto-adjust — feedback of use raises pain on hit chunks.

The `pain` field on `chunks` is an explicit severity in [0.1, 1.0] (0.1 trivial, 1.0 production outage). It can be set explicitly by the author, but it also drifts upward implicitly: chunks that are *repeatedly* surfaced by retrieval and *accepted* by the operator (via the `feedback/` directory, whose `chunk_type = feedback` is never-decay) accumulate co-occurrence with high-pain entries during nightly consolidation, which feeds back into the salience formula. The result is that lessons learned from real incidents naturally rise to the top over time, while toy or low-severity content fades even when frequent. This is the inverse of pure recency- or frequency-based memory.

3.4.3 Salience decay continuous in background — `recency × pain × importance`.

The salience function lives in `src/lib/salience.ts` [14] (and is mirrored in the staged copy at `staged-1.7a/edits/salience.ts`). Its canonical formula is:

```
salience = recency × pain × importance
```

with components:

- **recency** in [0,1] — half-life-style decay over the chunk's `retention_days` window (`feedback`/`person` → never-decay → `recency = 1.0`; everything else decays per Table V8 [15]). Decay is *continuous*: it is recomputed at

[13] V8 schema typed retention defaults (in `chunks.retention_days`): `feedback` = 0 (never-decay), `person` = 0 (never-decay), `lesson` = 180d, `decision` = 365d, `project` = 365d, `team` = 120d, `daily` = 90d, `pending` = 30d, `graph_node` = 60d, fallback = 90d. See `staged-1.7a/edits/salience.ts:46-56` (`DEFAULT_RETENTION_BY_TYPE`) and `CLAUDE.md` §"Schema v10".

[14] `src/lib/salience.ts` — canonical implementation of `salience = recency × pain × importance`, mirrored in the staged copy at `staged-1.7a/edits/salience.ts` (lines 1–80 contain the module docstring spelling out the formula, the three-state mode gate, and the per-type retention defaults).

[15] V8 schema typed retention defaults (in `chunks.retention_days`): `feedback` = 0 (never-decay), `person` = 0 (never-decay), `lesson` = 180d, `decision` = 365d, `project` = 365d, `team` = 120d, `daily` = 90d, `pending` = 30d, `graph_node` = 60d, fallback = 90d. See `staged-1.7a/edits/salience.ts:46-56` (`DEFAULT_RETENTION_BY_TYPE`) and `CLAUDE.md` §"Schema v10".

every retrieval, so there is no batch step that "ages" memory — aging is a property of the read path.
- **pain** in [0,1] — severity as described in §3.4.2.
- **importance** in [0,1] — `chunk_type` / `source_type` / tier signal (manual mapping; e.g., `decision` and `lesson` rank above `daily`).

The mode of operation is gated by `NOX_SALIENCE_MODE`: `shadow` (default — compute and log to `/api/health.salience` but do not apply to retrieval rankings), `active` (apply as an additive delta in [-0.5, +0.5] on top of RRF), and `off` (short-circuit to 0 for ablation experiments). This three-state gate is the operational realization of *shadow discipline* (§4 and §5).

3.4.4 Reflect pipeline — batch nightly cross-session synthesis.

The `reflect` command (CLI / MCP / `POST /api/reflect`, backed by `src/reflect.ts` and cached via `getReflectCacheStats()` exposed in `/api/health.reflectCache` [16]) runs a batch synthesis pass over recent chunks. Its job is *not* to retrieve answers for the user but to produce cross-session lessons: it queries hybrid search for a topic, asks Gemini to summarize the patterns observed across the result set, and writes the summary back as a `feedback` or `lesson` chunk with the confidence default for derived chunks (see `CONFIDENCE_DERIVED` in `docs/CONFIGURATION.md`). Because reflect results are themselves cached (LRU, exposed via `/api/health.reflectCache.entries`), expensive synthesis is amortized across repeated queries.

3.4.5 Consolidate — nightly orchestration that ties it together.

Nightly consolidation (23:00, 5-minute stagger across agents — see §3.2) is the orchestration layer that runs reflect for high-pain topics, runs crystallize for sufficiently aged `pending` items, and syncs the resulting durable entities to the topic files (`decisions.md`, `lessons.md`, `people.md`, `projects.md`). Like crystallize, it goes through `withOpAudit()` with a pre-op `VACUUM INTO` snapshot. This is the daily heartbeat of self-evolution.

Together, these four mechanisms make nox-mem a memory that *grows along the gradient of operator pain*: chunks that hurt the operator (production outages, lost work, repeated mistakes) survive decay, get promoted from pending to lesson, accumulate co-occurrence weight, and rank progressively higher — without retraining a model and without trusting a closed evolution loop. Every step is visible by opening `nox-mem.db` in `sqlite3` and inspecting `ops_audit`, `chunks.pain`, `chunks.retention_days`, and the entity files.

---

[16] `src/reflect.ts` exporting `reflect()` and `getReflectCacheStats()` (confirmed in `staged-1.6/edits/api-server.ts:12`). Cache statistics surfaced in `/api/health.reflectCache`. See also `docs/POSTMAN.md` for the API contract.

4. Hybrid Search System

4.1 Architecture

Search combines three complementary retrieval methods:

Layer 1 — FTS5 BM25 (Keyword)

SQLite FTS5 with porter unicode61 tokenizer provides fast keyword matching. Results are scored using BM25 with column weights (chunk_text: 1.0, source_file: 0.5, chunk_type: 0.5). Post-retrieval boosting applies:

- Type boost: `decision` and `lesson` chunks receive 2.0x multiplier (higher signal-to-noise ratio)
- Recency boost: Chunks from the last 7 days receive 1.5x multiplier

The query sanitizer strips special characters but preserves hyphens for compound terms (e.g., "nox-mem").

Layer 2 — Gemini Semantic (Vector)

Each chunk is embedded using Google's gemini-embedding-001 model (3072 dimensions) with task type RETRIEVAL_DOCUMENT. Query embeddings use task type RETRIEVAL_QUERY for asymmetric similarity optimization.

Vectors are stored in sqlite-vec virtual tables. Retrieval uses cosine distance with a map table (vec_chunk_map) bridging vec_chunks rowids to chunks.id values due to sqlite-vec's rowid-only constraint.

Scoring normalizes distances to a 0-10 scale with type and recency boosting (1.5x and 1.2x respectively, lower than FTS5 to avoid double-boosting in fusion).

Layer 3 — Reciprocal Rank Fusion (RRF)

FTS5 and semantic results are merged using RRF with k=60:

```
RRF_score(d) = Σ 1/(k + rank_i(d))
```

Documents appearing in both result sets receive combined scores, marked as `match_type: "hybrid"`. Content-prefix deduplication (first 50 characters) prevents near-duplicate results.

4.2 Performance Characteristics

The hybrid approach provides significant quality improvements over single-method search:

Query	FTS5 Only	Hybrid	Analysis
"qual o proximo passo"	0 results	ROADMAP + PHASE-3	Semantic captures intent without keyword match

Query	FTS5 Only	Hybrid	Analysis
"nox-mem"	0 results	decisions.md + docs	Vector bypasses tokenizer hyphen issues
"quem e o Toto"	people.md	people.md + TEAM_MEMORY	RRF combines exact match + semantic context

4.3 Cross-Agent Search

The `crossSearch()` function opens all 7 databases in read-only mode, executes FTS5 queries in each, and merges results with agent attribution. Deduplication uses content-prefix comparison to handle shared documents that appear across multiple agent databases.

---

5. Empirical Evaluation — Wave A G5 V3 (May 2026)

> **Headline (canonical, 2026-05-19):** A8 full stack with active salience reaches **nDCG@10 = 0.6237** on the entity-flavored golden set (n=100), a **+78.8% relative improvement over the G3 baseline (0.3488)** measured prior to Wave A deployment, and **+9.4% over the mid-deployment G4 checkpoint (0.5702)**. The peak ablation isolating `section_boost` alone (A3) reaches 0.6228, recovering **99.85% of the full stack** — section-aware ranking is the dominant driver of the lift.

5.1 Setup

The evaluation uses an entity-flavored golden set of 100 queries (`entity-eval.db`), curated from production usage to exercise the V10 schema's `section` and `pain` dimensions. Configurations are toggled via environment-variable feature gates (`NOX_SALIENCE_MODE`, `NOX_DISABLE_TIER_BOOST`, `NOX_ENABLE_TIER_BOOST`, `NOX_DISABLE_SECTION_BOOST`, etc.), allowing isolation of individual ranking components without code changes between runs. All measurements occur post-deployment of PRs #150 (salience formula + tier_boost off-by-default), #151 (source_type backfill of 67,949 chunks), and #153 (search wiring). Reported nDCG@10 follows the standard TREC formulation (gain by relevance, log-position discount).

Progression vs prior ablation generations:

Generation	Date	A8 nDCG@10	Δ vs G3 baseline	Notes
G3 baseline (pre-Wave A)	2026-05-15	0.3488	—	Multiplicative salience, tier_boost on, section_boost only via legacy code path
G4 mid-deployment	2026-05-18	0.5702	+63.5%	Salience aditivo wired but `active < shadow` puzzle observed
G5 V3 canonical	**2026-05-19**	**0.6237**	**+78.8%**	Wave A fully deployed; reversal `active > shadow` cravado

The four sub-claims below decompose the +78.8% total into measurable contributions; the full G5 V3 matrix (12 configurations) is archived in `audits/` and HANDOFF.md (`#g5-v3-matrix-2026-05-19`).

5.2 Claim 1 — Additive salience outperforms multiplicative

The Wave A formula replaces the legacy multiplicative `salience = recency × pain × importance` with a weighted-additive form (PR #150):

```
salience = W_IMPORTANCE·importance + W_RECENCY·recency + W_PAIN·pain + W_ACCESS·access_score
W_IMPORTANCE = 0.55  W_RECENCY = 0.15  W_PAIN = 0.10  W_ACCESS = 0.20
```

Result. With `NOX_SALIENCE_MODE=active`, A8 reaches 0.6237 vs. 0.6155 with `shadow` (A7) — a +1.3% lift and the reversal of the G4 puzzle, where shadow had outranked active. The multiplicative form concentrated 99.7% of chunks in the [0.05, 0.40] salience range, dominated by 90.67% of chunks at the default `pain = 0.2` and 99.76% of chunks with `recency in [7, 30]` days; small differences in any factor were swallowed by the product. The additive form exposes each dimension proportionally to its calibrated weight, preserving signal from pain spikes and importance heuristics without requiring all three factors to be simultaneously non-default.

5.3 Claim 2 — `section_boost` is the moat (99.85% of the gain)

Isolating `section_boost` alone (A3 ablation: section enabled, tier off, source_type off, salience shadow) yields **nDCG@10 = 0.6228 = 99.85% of A8's full-stack 0.6237**. The V10 schema multipliers — `compiled = 2.0`, `frontmatter = 1.5`, `timeline = 0.8`, legacy = 1.0 — together with the entity-file format introduced in v3.7 (769 entity files × 3 sections ~= 2,307 boost-bearing chunks) explain the majority of the headline improvement.

The negative control A11 (full stack minus `section_boost`) drops to 0.5646, **−9.5% relative to A8**, confirming the contribution is not redundant with semantic embeddings or RRF fusion. This is the architectural pivot the paper's narrative rests on: section-aware boosting over an entity-file canonical form is the load-bearing component, not the multiplicative salience formula that the v1 paper draft over-emphasized.

5.4 Claim 3 — `tier_boost` off-by-default is the correct calibration

Isolated, `tier_boost` (boost for `chunks` flagged as `tier='core'`) is actively harmful: A6 (tier only, no other boosts) reaches 0.4059, **−21% versus the no-boost baseline 0.5126**. Even integrated into the full stack, A9 (full + tier enabled) drops to 0.5884, **−5.7% versus A8**. Inspection of the corpus reveals the cause: `tier='core'` chunks account for only 3.96% of the corpus and consist of memory-system internals (lifecycle docs, schema metadata, operational runbooks) rather than user content — over-promoting them displaces directly-relevant entity facts.

PR #150 therefore makes tier_boost **off by default** via `NOX_DISABLE_TIER_BOOST=1`, with an explicit `NOX_ENABLE_TIER_BOOST=1` opt-in for callers who want the legacy behavior. The default reflects the calibration this evaluation establishes; the opt-in preserves backward compatibility for downstream pipelines that depend on it.

5.5 Claim 4 — `source_type` backfill recovery

Pre-backfill, **67,949 chunks (98.48% of the corpus)** carried `source_type = NULL`, rendering the `SOURCE_TYPE_BOOST` map inert at search time regardless of the configured multipliers. PR #151 backfills 11 canonical keys (`entity`, `lesson`, `skill`, `project-doc`, `command`, `legal-template`, `personal-doc`, `session`, `note`, `external`, `other`, `ocr-cache`) by classifying each chunk via deterministic path/prefix rules under `withOpAudit()` (audit_id = 118), preserving the 1,046 chunks already marked `external` (1.52%). The post-backfill distribution skews to `personal-doc` (32.74%), `skill` (19.89%), and `session` (16.95%), reflecting the lived shape of the operational corpus.

In the G5 V3 matrix, A5 (source_type only) and A10 (full minus source_type) both score 0.6237 — identical to A8 — confirming the `SOURCE_TYPE_BOOST` map remained **inert by key mismatch**, not by data absence. PR #154 (merged

2026-05-20) updated the boost map to the new keys (calibration ranging from `entity = 2.0` for high-curation chunks down to `ocr-cache = 0.7` for low-signal scanned material).

The G8 ablation (2026-05-20, PR #177) re-ingested an isolated `entity-eval-v2.db` (500 chunks) with source_type values deterministically remapped to prod-consistent vocabulary (`entity_file → entity`, `event_log → lesson`, `session_summary → session`), achieving 100% key-match with the SOURCE_TYPE_BOOST map. Results:

Config	nDCG@10	Δ vs A0	Verdict
A0 (no boosts)	0.4816	baseline	—
A5 (source_type only)	**0.4944**	**+2.66%**	LIVE — boost contributes when keys match
A8 (full canonical)	0.5798	+20.4%	full stack
A10 (full minus source_type)	0.5845	+21.4%	source_type *removed* from full stack

A5 > A0 by +2.66% empirically validates that `SOURCE_TYPE_BOOST` contributes to ranking when source_type values match the map keys. However, A8 < A10 by −0.81% reveals **redundant double-boost** when `section_boost` (e.g., `compiled = 2.0` for entity-file truth sections) and `SOURCE_TYPE_BOOST` (e.g., `entity = 2.0` for the same chunks) stack on identical chunks, over-promoting at the cost of top-K diversity. Per-category, the redundancy manifests as a −3.5 pp regression on open-domain queries and a +1.4 pp gain on multi-hop.

The **G9 ablation** (2026-05-20, against the prod-flavored `g5.db` 68k corpus, PR planned) **reproduces and amplifies** both findings — at production scale the boost contribution and the redundancy are both **5× larger in magnitude** than in the synthetic G8 set:

Config	G8 (n=500)	G9 (n=68,995)	Δ G9 magnitude vs G8
A0 (no boosts)	0.4816	0.4108	smaller baseline (prod diversity)
A5 (source_type only)	**+2.66% vs A0**	**+14.2% vs A0**	**5× larger**
A8 vs A10 (redundancy)	**−0.81%**	**−2.6%**	**3× larger**

The G9 data **structurally validates** the resolution path of mutual-exclusion logic (PR #182, merged 2026-05-20): when a chunk has `section in {compiled, frontmatter, timeline}` populated (entity-file structural metadata), the `source_type_boost` is gated to `0` to prevent stacking on top of `section_boost`. The mutex is rollback-gated via `NOX_DISABLE_MUTEX_SECTION_SOURCE_TYPE=1`.

The **G10 ablation** (2026-05-20, against `g9.db` 69,495 chunks) validates the mutex in production conditions:

Config	nDCG@10	MRR	R@10
A8' (mutex active, default)	**0.5478**	**0.5967**	0.6183
A8 (mutex disabled, rollback flag)	0.5435	0.5813	0.6333
Δ mutex effect	**+0.79%**	**+2.65%**	−2.4%

The mutex recovers ~46% of the A10 − A8 gap (where A10 fully removes `source_type_boost`) without removing the signal entirely. The per-metric pattern — MRR ↑ (top-1 quality) and R@10 ↓ (diversity) — surfaces a deliberate trade-off: the mutex improves precision at the cost of some recall breadth, with net positive on the weighted nDCG@10.

The **G10b per-category breakdown** (2026-05-21, same DB, n = 100 across 5 categories) reveals which query types absorb the trade-off:

Category	n	nDCG@10 Δ%	MRR Δ%	R@10 Δ%	Verdict
single-hop	20	**+8.22%**	**+13.20%**	0%	strong win
open-domain	20	**+2.42%**	**+5.56%**	0%	win
multi-hop	20	−3.95%	−2.70%	**−6.02%**	regression
adversarial	20	−2.95%	−5.88%	0%	regression
temporal	20	n/a	n/a	n/a	degenerate corpus gap

The aggregate Δ (+0.43% nDCG, +0.82% MRR) is consistent in direction with the G10 measurement (+0.79% / +2.65%) but attenuated in magnitude — within harness noise at n = 100, suggesting the deploy-time figure was on the upper end of a noisy distribution. Substantively: the mutex is a **single-hop optimizer with open-domain side benefits**, balanced against a multi-hop chain-traversal regression of −6.02% R@10. Net retrieval value (+0.0616 nDCG abs gain) exceeds losses (−0.0498 nDCG abs), so the mutex stays deployed; the multi-hop regression is documented as a follow-up candidate for **conditional mutex** (active only when `query_entities ≤ 1`).

The **G11 trim ablation** (2026-05-20, same DB) tested whether trimming the top SOURCE_TYPE_BOOST values (`entity: 2.0 → 1.3`, `lesson: 1.8 → 1.2`) could provide additive benefit over the mutex:

Config	nDCG@10	MRR
Canonical (entity=2.0, lesson=1.8)	**0.5376**	**0.5843**
Trim (entity=1.3, lesson=1.2)	0.5337	0.5751
Δ	**−0.73%**	**−1.58%**

Trim is **rejected**. The mutex already zeroes `sourceTypeDelta` where redundancy occurs (chunks with both `section` and high source-type); the `entity = 2.0` still fires legitimately on legacy non-compiled chunks where the mutex does not trigger, and trimming kills that residual signal. The single-hop category suffered worst (−4.62% nDCG, −7.40% MRR), confirming the mutex resolves redundancy **precisely**, while a global trim over-corrects. The boost stack settles at the canonical configuration: `section_boost × source_type_boost (mutex-gated) × additive salience v2`.

The **G10c per-style breakdown** (2026-05-21, same DB, n = 100 across 2 styles — the dataset distinguishes `keyword` vs `natural-language` rather than the paraphrase/literal axis the spec anticipated) cuts the same data along a different dimension to test whether mutex behavior is style-conditional:

Style	n	nDCG@10 Δ%	MRR Δ%	R@10 Δ%	Verdict
natural-language	50	**+1.56%**	**+3.86%**	−1.62%	mutex helps
keyword	50	−0.72%	−2.27%	−1.06%	mutex slightly hurts

The aggregate effect (+0.43% nDCG, +0.82% MRR — identical to G10b because G10c reuses the same A8 active vs A8 disabled detail JSONs and re-buckets) is entirely carried by the natural-language subset. The keyword bucket is a small drag, within the noise floor. Two cross-cuts (style × category) surface as notable outliers: NL × single-hop (+13.83% nDCG, +21.32% MRR — the biggest individual win in the data) and keyword × adversarial (−5.35% nDCG, −10.0% MRR — the only delta crossing the 5% regression threshold, n = 10). Multi-hop suffers ~= −4% across both styles, confirming the regression is **style-agnostic** and motivating the conditional-mutex follow-up rather than a style-specific routing.

Triangulated across G10 (deploy figure +0.79% / +2.65%), G10b (per-category +0.43% / +0.82%), and G10c (per-style +0.43% / +0.82%), the mutex effect is consistent in direction and on the lower end of the original deploy measurement in magnitude — the deploy figure sat on the upper tail of a noisy distribution rather than reflecting a structural shift. The architectural conclusion stands: **keep the mutex deployed at the per-chunk level; address the**

multi-hop chain-traversal regression via a conditional gate keyed on `query_entities` — executed in G10d below.

The **G10d ablation** (2026-05-21, same `g9.db` corpus, 69 495 chunks, 15 612 `kg_entities`) tests a conditional variant of the Hard Mutex: the per-chunk gate is suppressed when the incoming query matches two or more entities in the KG, preserving the full boost stack for multi-entity chain-traversal queries while keeping the mutex active for single-entity and entity-free queries. The mechanism relies on `query-entity-count.ts`, which performs a greedy longest-match scan over `kg_entities.name` at query time; the count drives `NOX_MUTEX_QUERY_ENTITY_THRESHOLD` in `search.ts`. Four configurations were run against the same 100-query golden set (n = 100, 5 categories × 2 styles × 10), each on an isolated endpoint (port 18803) with salience active, ~13 min total VPS time:

Config	Description	nDCG@10	MRR	R@10	Δ%nDCG vs A8'	Δ%MRR vs A8'
A8'	G10 Hard Mutex always-on (prod baseline)	0.5502	0.5992	0.6183	—	—
A8d-1	Conditional, threshold=1	0.5467	0.5856	0.6333	−0.64%	−2.27%
A8d-2	**Conditional threshold=2**	**0.5577**	**0.6074**	**0.6233**	**+1.35%**	**+1.37%**
A8 off	Mutex fully disabled (control)	0.5438	0.5806	0.6333	−1.17%	−3.10%

The headline result is **A8d-2 (threshold=2) wins on all three primary metrics** over the A8' G10 baseline. A8d-1 regresses on nDCG and MRR, pointing to a critical entity-density effect: with 15 612 entities in `kg_entities` — 40× the 402 initially estimated at design time — a threshold of 1 is effectively always met, collapsing the conditional back to the constant-off regime (no mutex). Threshold=2 is the minimum noise filter that restores actual conditionality at current entity density.

The per-category breakdown reveals the mechanism of recovery:

Category	n	nDCG@10 Δ% vs A8'	MRR Δ% vs A8'	R@10 Δ% vs A8'	Verdict
multi-hop	20	**+1.58%**	0.00%	**+3.75%**	recovery — chain signal preserved
adversarial	20	**+3.04%**	**+6.25%**	0.00%	recovery — distractor suppression improves
open-domain	20	**+2.92%**	**+1.59%**	0.00%	extends G10b win
single-hop	20	−3.26%	−4.43%	0.00%	trade-off
temporal	20	n/a	n/a	n/a	degenerate corpus gap (unchanged)

Multi-hop recovers because queries naming two or more entities (e.g., entity + associated event or related entity) now reach top-K with the full `section_boost × source_type_boost` stack active, enabling intermediate chain chunks to surface. Adversarial recovery is the strongest signal: adversarial queries in the golden set tend to mention three or more entity names as distractors, pushing `query_entity_count >= 2` and gating the mutex; the full boost stack then differentiates gold from distractors more effectively than the mutex-flattened ranking. The single-hop trade-off is real — A8d-2 nDCG drops −3.26% vs A8' — but is bounded: single-hop performance against the pre-mutex baseline (G10b mutex_disabled) is still **+3.31% nDCG / +7.78% MRR** (absolute: 0.5470 vs 0.5295 disabled). The conditional layer trades peak single-hop precision for materially better worst-case behavior across multi-hop and adversarial categories.

Latency is unaffected: P95 spread across all four configs is 2558–2573 ms (0.6% variance), consistent with the `query-entity-count` hot path operating at sub-millisecond cost when the entity index is warmed.

Decision D51 verdict: ACTIVE-T2. A8d-2 meets 6 of 8 evaluated criteria (aggregate nDCG/MRR, multi-hop nDCG/R@10, open-domain nDCG, adversarial nDCG). Single-hop nDCG and MRR are the two fails — both against the

A8' baseline that represents the maximal single-hop state — and both remain strictly positive against the pre-mutex baseline. The aggregate net of +1.35% nDCG / +1.37% MRR over the G10 baseline justifies accepting the single-hop dilution. The canonical boost stack in production now reads: `section_boost × source_type_boost (Hard Mutex gated by query_entity_count ≤ 2) × salience v2 additive`.

The G10d conditional gate was deployed to production on 2026-05-21 via systemd environment drop-in (`NOX_MUTEX_QUERY_ENTITY_THRESHOLD=2`). A smoke test across three query archetypes confirmed correct behavior: a single-entity query applied the mutex as expected; a multi-entity query (count >= 2) returned an `entity::compiled` chunk at rank 1, confirming the mutex was suppressed and the full boost stack served the chain; a no-entity query bypassed the mutex entirely. Zero errors were recorded in `journalctl` post-restart. Three rollback paths are documented — disabling only the conditional layer (preserving G10 hard mutex), disabling the entire mutex, and removing the drop-in — each executable in under five minutes.

Triangulated across G10 (+0.79% nDCG deploy measurement), G10b (per-category breakdown, aggregate +0.43%), G10c (per-style breakdown, aggregate +0.43%), and G10d (conditional gate, aggregate +1.35%), the mutex evolution follows a consistent trajectory: the per-chunk hard mutex provided a net positive but introduced multi-hop and adversarial regressions; the conditional layer recovers those regressions at the cost of moderate single-hop dilution, with the aggregate strictly improving at each step. The final deployed configuration is the most balanced across query-category diversity the series has measured.

5.6 Production deployment and observability

The G10d conditional Hard Mutex with `NOX_MUTEX_QUERY_ENTITY_THRESHOLD=2` é a configuração canônica em produção desde 2026-05-21. O drop-in está em `/etc/systemd/system/nox-mem-api.service.d/override.conf`, e três rollback paths permanecem documentados — desabilitar apenas a camada condicional (preservando o G10 hard mutex), desabilitar o mutex inteiro via `NOX_DISABLE_MUTEX_SECTION_SOURCE_TYPE=1`, ou remover o drop-in — cada um executável em menos de cinco minutos. O modo `NOX_SALIENCE_MODE=active` (formulação aditiva v2) também está deployado em produção, consistente com a Claim 1 da §5.2; o modo `shadow` permanece disponível como fallback para A/B comparisons, mas o canonical runtime usa `active`.

A camada de observabilidade F10 (Foundation observability dashboard, decisão D53, 2026-05-21) acompanha os dois deploys em produção. **Phase A** (`/observability/health.html`) expõe três endpoints — `/api/observability/health`, `/api/observability/recent-ops`, `/api/observability/canary-tail` — com polling de 30s sobre status do serviço, últimas operações destrutivas registradas em `ops_audit` (status enum `started | success |`

`failed | crashed`), e o tail das execuções do cron de canary. **Phase B** (`/observability/evals.html`) consome `/api/observability/evals` lendo `audits/data-G*/`, renderizando line charts com Chart.js sobre as séries G3 → G4 → G5 V3 → G8 → G9 → G10 → G10b → G10c → G10d com gate annotations (D43 threshold >=+15% nDCG@10, D48 close, D51 verdict ACTIVE-T2). Ambas as fases passaram smoke tests no deploy (6/6 e 5/5 respectivamente) e estão acessíveis via Tailscale tunnel; o stack permanece lean (vanilla JS + Chart.js CDN, sem Prometheus/Grafana/time-series DB adicional). A leitura é em tempo real sobre o `nox-mem.db` canônico — qualquer regressão pós-deploy aparece nos charts dentro do próximo ciclo de polling.

5.7 Honest characterization

The +78.8% headline is decisive for the "Pain-weighted hybrid memory" framing in the sense that pain is one of four additive salience components and the full additive formulation outperforms the legacy multiplicative one. It is **not** decisive for "pain as a standalone retrieval signal in hybrid mode" — that question is addressed in the companion arXiv draft (`paper/publication/paper-draft-sec4-7.md` §5.5, E10 pain ablation: directional but not significant, $\Delta = +0.0065$, 95% CI $[-0.0143, +0.0338]$, $n = 31$ on the prior R01c-v1.1 corpus). The Wave A measurement validates the architectural choices around section-aware ranking and additive salience composition; per-dimension causal attribution of pain alone awaits the post-PR-#154 ablation generation and a corpus with broader pain distribution than the current 90.67% default.

A G10d evolution further refines the architectural conclusion: the canonical boost stack `section_boost × source_type_boost (Hard Mutex gated by query_entity_count ≤ 2) × salience v2 additive` deployed em 2026-05-21 trata a redundância identificada em G8/G9 sem zerar o sinal completo, e recupera regressões multi-hop (+1.58% nDCG, +3.75% R@10) e adversarial (+3.04% nDCG, +6.25% MRR) ao custo de uma diluição contida em single-hop. A trajetória G3 → G4 → G5 V3 → G8 → G9 → G10 → G10b → G10c → G10d demonstra disciplina de ablation: cada generation isolou um componente ou condição, e cada decision (D43 gate, D48 saga close, D51 ACTIVE-T2) está triangulada por código (PRs #150/#151/#153/#154/#177/#181/#182/#198), audits (`audits/data-G*/`), e a camada F10 que torna o resultado verificável a qualquer momento em produção.

---

6. Q4 COMPARISON — Cross-System Benchmarking (Pre-registered)

> **Status (atualizado 2026-05-24 ~22h BRT — FINAL):** Sat 2026-05-24 FINAL closure. **4/6 systems com dados reais.** Decision A aprovada: ship 4/6 (Zep [GATED] por OpenAI embedding requirement; EverMind-AI [SKIP] por repo 404

confirmado PR #281). nox-mem headline: nDCG@10=0.6380 (Gemini hybrid) / 0.3753 (FTS5-only). mem0 (500-chunk cap) + agentmemory (1401-chunk cap) + Letta (partial 1/5 smoke) medidos com caveats de corpus. Canonical 100-query run deferred Sun 2026-05-25 com corpus uniforme sem cap. Princípios (§6.5), anti-cherry-pick (§6.6) e pre-registration (§6.7) imutáveis. Nota: "Sat 2026-05-24 partial; canonical full-corpus run Sun 2026-05-25." Refs: `[[q4-real-numbers-sat-2026-05-24]]` · `[[q4-partial-cross-system-sat-2026-05-24]]`.

6.1 Methodology summary

A §6 cobre a comparação cross-system entre nox-mem e cinco sistemas competidores de memória persistente para agentes de IA. O execution plan completo está documentado em `specs/2026-05-23-Q4-comparison-execution-plan.md` (pre-registered 2026-05-23, antes do run de Sat 2026-05-24). Os princípios de comparação (§6.5), as garantias anti-cherry-pick (§6.6), e a pre-registration formal (§6.7) são cravados nesta seção antes da execução; somente as tabelas de §6.2/§6.3/§6.4 recebem números após o run. O objetivo é satisfazer o gate D43 (`docs/DECISIONS.md`) — nox-mem em top-3 em >=2 das 4 métricas chave (nDCG@10, R@10, MRR, latência) — destravando a GTM Phase 2.

6.2 Competitors

A escolha dos cinco competidores prioriza stars no GitHub, atividade recente de commits e overlap funcional com o escopo do nox-mem. Versões são cravadas pré-execução para reprodutibilidade.

System	Repo	Install path	Version pinned	Default config
Mem0	`mem0ai/mem0`	`pip install mem0ai`	`[PENDING canonical run − adapter under setup]`	OpenAI embeddings + Chroma vector store
Zep	`getzep/zep`	Docker compose (zep + postgres)	`[PENDING canonical run − adapter under setup]`	Local self-host mode

System	Repo	Install path	Version pinned	Default config
Letta (ex-MemGPT)	`letta-ai/letta`	`pip install letta`	`[PENDING canonical run — adapter under setup]`	SQLite backend
agentmemory	`rohitg00/agentmemory`	iii-engine runtime	`[PENDING canonical run — adapter under setup]`	Stack-bridge mode
EverMind-AI	EverOS published bench	repo clone	`[PENDING canonical run — adapter under setup]`	Native CLI

Cada sistema roda com sua configuração default publicável (princípio §6.5.3): nenhum competidor é tunado adversarialmente.

6.3 Per-system per-dataset results

Tabela canônica cross-system × cross-dataset. K cutoff fixado em 10 em todos os sistemas; latência medida externamente (wall clock around adapter call); custo derivado dos logs por-sistema (API calls × pricing publicado).

Sat 2026-05-24 partial cross-system smoke (20 queries combined, dry-run-sample, eval-isolated DB). nox-mem: full corpus (6.822 chunks = 5.882 LoCoMo + 940 LongMemEval). mem0: **500-chunk corpus cap** por cost-control ($0.10 ingest cost estimado; ~8% do corpus completo). Caveat crítico: os números do mem0 refletem um corpus significativamente menor — interpretação no parágrafo abaixo.

System	n	Corpus chunks	nDCG@10	R@10	MRR	p50 (ms)	avg (ms)	Gold hits
nox-mem	20	6.822 (full)	0.6380	0.5417	**0.3700**	**8**	**9**	**13/20 (65%)**
mem0 (500-cap)	20	500 (~8%)	**0.8569**	0.2500	0.1167	273	288	3/20 (15%)

Per-dataset gold-hit breakdown do nox-mem smoke: **LoCoMo 7/10 (70%) · LongMemEval 6/10 (60%)**.

Interpretação do trade-off (honestidade obrigatória):

Os dois sistemas exibem perfis opostos. nox-mem, com corpus completo (6.822 chunks, ingest local zero-custo), produz **4× maior hit-rate** (65% vs 15%) e **MRR 3× melhor** (0.37 vs 0.12) — o primeiro hit relevante chega antes em nox-mem. A latência de nox-mem é **30× mais rápida** (8ms p50 vs 273ms p50), reflexo da busca local vs chamadas à API mem0.

mem0, operando sobre apenas 500 chunks (~8% do corpus), exibe **nDCG@10 superior** (0.86 vs 0.64): os poucos hits que retorna tendem a ser top-ranked, produzindo alta concentração de relevância nas primeiras posições. Isso é um artefato de corpus window menor — com janela restrita, o sistema tem menos competição entre resultados candidatos, o que infla o nDCG per-se mas mascara a cobertura real (R@10 = 0.25 vs 0.54). Em produção com corpus completo e mesmo custo de ingest, a relação nDCG pode inverter; o run canônico (corpus uniforme, sem cap) será o árbitro desta hipótese.

Resumo executivo: **nox-mem ganha em cobertura (hits), velocidade (latência), e first-hit quality (MRR). mem0 ganha em concentração de relevância por-resultado (nDCG@10) dentro de uma janela de corpus menor.** Corpus cap de 500 chunks para mem0 é cost-control explícito — $0.10 estimado vs zero-cost local; dados iguais de corpus revertem parcialmente o nDCG gap.

O smoke não disaggregou `nDCG@10` por dataset (combined-only) — desagregação canônica vem no run completo. Os números a seguir são da execução canônica que ainda está em curso.

Sat 2026-05-24 FINAL — 4/6 systems with real data. Decision A: ship 4/6 (Zep gated, EverMind skipped). Canonical 100-query run deferred Sun 2026-05-25.

LongMemEval n=100 (canonical — pending Sun 2026-05-25):

System	nDCG@10	R@10	MRR	p50 (ms)	p95 (ms)	p99 (ms)	Cost/query (USD)
nox-mem	`[pending Sun canonical]`	`[pending]`	`[pending]`	`[pending]`	`[pending]`	`[pending]`	~$0.00 (local)

System	nDCG@10	R@10	MRR	p50 (ms)	p95 (ms)	p99 (ms)	Cost/query (USD)
Mem0	[pending — Sun canonical — full corpus, no 500-cap]	[pending]	[pending]	[pending]	[pending]	[pending]	~$0.10+ ingest
agentmemory	[pending — Sun canonical — full corpus, no 20%-cap]	[pending]	[pending]	[pending]	[pending]	[pending]	~$0.00
Letta	[pending — partial only; agent-loop arch]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
Zep	[FALHA: [GATED] — OpenAI embedding requirement + adapter rewrite needed; deferred post-launch]	—	—	—	—	—	~$0.02 est.

System	nDCG@10	R@10	MRR	p50 (ms)	p95 (ms)	p99 (ms)	Cost/query (USD)
EverMind-AI	[FALHA: [SKIP] — repo EverOS-AI/EverMind-AI re-turns 404; con-firmed 2026-05-24 PR #281]	—	—	—	—	—	—

LoCoMo full (canonical — pending Sun 2026-05-25):

System	nDCG@10	R@10	MRR	p50 (ms)	p95 (ms)	p99 (ms)	Cost/query (USD)
nox-mem	[pending Sun canon-ical]	[pending]	[pending]	[pending]	[pending]	[pending]	~$0.00 (local)
Mem0	[pending Sun canon-ical — full cor-pus, no 500-cap]	[pending]	[pending]	[pending]	[pending]	[pending]	~$0.10+ ingest

System	nDCG@10	R@10	MRR	p50 (ms)	p95 (ms)	p99 (ms)	Cost/query (USD)
agentmemory	[pending Sun canonical — full corpus, no 20%-cap]	[pending]	[pending]	[pending]	[pending]	[pending]	~$0.00
Letta	[pending — partial only; agent-loop arch]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
Zep	[FALHA: [GATED] — OpenAI embedding requirement + adapter rewrite needed; deferred post-launch]	—	—	—	—	—	~$0.02 est.

System	nDCG@10	R@10	MRR	p50 (ms)	p95 (ms)	p99 (ms)	Cost/query (USD)
EverMind-AI	[FALHA: [SKIP] — repo EverOS-AI/EverMind-AI re-turns 404; con-firmed 2026-05-24 PR #281]	—	—	—	—	—	—

Zep e EverMind-AI são reportadas com `[FALHA: <razão>]` explícito em vez de omitidas — consistente com §6.6 (anti-cherry-pick). O run canônico Sun 2026-05-25 atualiza as células `[pending]` para os 4 sistemas restantes com corpus uniforme (sem cap). Ref: `[[q4-real-numbers-sat-2026-05-24]]`.

6.4 Per-category breakdown

Decomposição por categoria de query do LongMemEval. nox-mem reporta as seis categorias canônicas; competidores reportam idem onde a categoria está presente no dataset original.

Category	n	nox-mem nDCG@10	Mem0	Zep	Letta	agentmemory	EverMind-AI
single-hop	[PENDING canon-ical]	[PENDING canon-ical]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
multi-hop	[PENDING canon-ical]	[PENDING canon-ical]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
temporal	[PENDING canon-ical]	[PENDING canon-ical]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
adversarial	[PENDING canon-ical]	[PENDING canon-ical]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]

Category	n	nox-mem nDCG@10	Mem0	Zep	Letta	agentmemory	EverMind-AI
open-domain	[PENDING canon-ical]	[PENDING canon-ical]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
numeric	[PENDING canon-ical]	[PENDING canon-ical]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]

O **smoke de Sat 2026-05-24** não disaggregou per-category (combined-only sobre 20 queries), portanto §6.4 inteira aguarda o run canônico de 100 queries × 2 datasets × 6 sistemas. Quando uma categoria não tem queries suficientes (n < 10) em algum dataset, a célula recebe `n/a` em vez de extrapolação, evitando o tipo de regressão que aparece em §5.5 (temporal `n/a` na G10b por corpus gap degenerado).

6.5 Fair-comparison principles

A comparação obedece a princípios padronizados pela literatura de benchmark publicado (EverMemBench, BEIR, MTEB):

1. **Corpus idêntico.** Todos os sistemas recebem o mesmo `chunks.text` ingerido via a API nativa de cada um. Nenhum sistema recebe versão "otimizada" do corpus.
2. **Eval set idêntico.** Mesmas queries, mesmos gold sets, mesmo random seed (`42` para shuffle do LongMemEval).
3. **Defaults nativos por sistema.** Cada competidor roda com sua configuração default publicável. Não tunamos competidores adversarialmente para perder; se default config é o que é publicado, é o que é avaliado.
4. **K cutoff fixo em 10.** Alguns sistemas defaultam para 5 ou 20; todos são forçados a `k=10` para comparabilidade.
5. **Embeddings provider nativo por sistema.** nox-mem usa Gemini 3072d; cada competidor usa seu provider default. Uma variação `all-Gemini` é planejada como side experiment opcional, deferred porque o smoke de Sat 2026-05-24 consumiu o time-box antes do experiment (`[deferred — Sun 2026-05-25 ou follow-up post-launch]`).
6. **Hardware uniforme.** Mesmo VPS (Hostinger 8 cores / 16 GB RAM), localhost between systems exceto chamadas a embeddings APIs externas.

Cada adapter passa um smoke test pré-run:

```
result = adapter.search("test query", k=5)
assert len(result) >= 1
assert all('id' in r and 'score' in r for r in result)
```

Adapter que falhar smoke é documentado como gap (`[FALHA: <razão>]`) em vez de omitido — consistente com §6.6.

6.6 Anti-cherry-pick statement

Para evitar viés de seleção retroativo:

- **Todas as 6 categorias reportadas.** Nenhuma é omitida porque o resultado é desfavorável.
- **Ambos os datasets reportados.** LongMemEval n=100 + LoCoMo full, lado a lado. Não escolhemos o que beneficia.
- **Latência worst-case reportada.** p50 + p95 + p99 explícitos. Não publicamos apenas p50.
- **Per-system per-category transparency.** A tabela de §6.4 expõe cada combinação; não há linha agregada que mascare um padrão.
- **Gaps documentados.** Sistemas que falharem setup recebem nota explícita; a comparação roda sem o sistema faltante, mas o gap é registrado em `docs/COMPARISON.md`.
- **Per-dataset breakdown explícita (PR #318, 2026-05-23 — rev3).** O run Gemini hybrid@500 revelou que o aggregate (0.0918) mascara um resultado por-dataset decisivo. Reportamos as três linhas explicitamente:

System	nDCG@10 (aggregate)	nDCG@10 (LoCoMo-only)	Corpus	Mode
nox-mem FTS5@500	0.0466	—	500 (cap)	FTS5-only
nox-mem Gemini hybrid@500	0.0918	**0.1835**	500 (cap)	FTS5 + Gemini + RRF
mem0@500	**0.1315**	0.1315	500 (cap)	LLM rewrite + embed

LoCoMo-only result (PR #318): nox-mem Gemini hybrid@500 = 0.1835 **supera** mem0@500 = 0.1315 em **+40%** na dimensão de memória conversacional. O aggregate (0.0918) fica abaixo de mem0 por um **artefato de corpus-ordering**: ao 500-chunk cap, os 5.882 chunks do LoCoMo esgotam o cap antes de qualquer chunk do LongMemEval ser ingerido — as 10 queries LongMemEval ficam com cobertura zero, zerando o nDCG desse dataset e puxando o aggregate para baixo. Hybrid

stack lift sobre FTS5@500: **+97%** (0.0466 → 0.0918), validando o valor arquitetural do stack mesmo em corpus esparso.

H2 finding (PR #311, mantido): FTS5-only@500 = 0.0466 vs mem0@500 = 0.1315 é **real e arquitetural** para o modo FTS5-only — LLM-rewriting do mem0 produz generalização semântica que FTS5 isolado não consegue. PR #318 mostra que o Gemini hybrid completo inverte esse resultado no escopo conversacional.

Disclosure obrigatória: o aggregate ±0.05 está dentro do intervalo inconclusivo para n=20. O árbitro definitivo é o run canônico full-corpus (corpus uniforme sem cap, LoCoMo + LongMemEval completos para todos os sistemas). **Phase 2 gate usa AMBOS** per-dataset + aggregate no run canônico — não apenas o número que favorece nox-mem.

Refs: `docs/COMPARISON.md §Apples-to-apples corpus-cap comparison`, PR #311, PR #318.

6.7 Pre-registration

A metodologia desta seção está cravada no `specs/2026-05-23-Q4-comparison-execution-plan.md` antes do run de Sat 2026-05-24. O **smoke de Sat 2026-05-24 15h30 BRT** preencheu a primeira linha de §6.3 (nox-mem combined: nDCG@10=0.6380, p50=8ms, gold-hit 13/20 em 20 queries dry-run-sample) e validou que o pipeline de retrieval funciona end-to-end em eval-isolated DB. O **partial cross-system smoke de Sat 2026-05-24 18h BRT** adicionou a linha mem0 (n=20, 500-chunk corpus cap): nDCG@10=0.8569, p50=273ms, gold-hit 3/20 (15%) — com interpretação explícita do trade-off coverage vs concentração em §6.3. O **run canônico** ainda está em curso e atualiza as linhas competidoras `[PENDING canonical run]` em §6.3 + a totalidade de §6.4 quando os 6 adapters estiverem prontos com corpus uniforme. Princípios (§6.5), anti-cherry-pick (§6.6) e a estrutura geral desta seção são imutáveis post-run. Qualquer ajuste metodológico identificado durante a execução é documentado como follow-up explícito em `docs/COMPARISON.md` em vez de retroagido aqui. Refs: `[[q4-smoke-sat-2026-05-24-real-numbers]]` · `[[q4-partial-cross-system-sat-2026-05-24]]`.

A decisão D43 (`docs/DECISIONS.md`) define o gate de aprovação: nox-mem em top-3 em >=2 das 4 métricas chave (nDCG@10, R@10, MRR, latência). Atendido o gate, GTM Phase 2 está destravada conforme `docs/ROADMAP.md` §7. Não atendido, a sessão de Sun 2026-05-25 produz um plano de remediação (ajustes pre-launch) em vez de launch direto.

6.8 Autonomy quantified — operational cost per memory system

The Q4 quality comparison (§6.3 – §6.6) reports retrieval *quality* under matched corpora. Operational *cost* — services, RAM, cold start, mandatory third-party

credentials, setup commands — is the second axis on which a memory system can be evaluated, and is the axis where the nox-mem Autonomy pillar [17] is most legible. Table 2 summarizes the steady-state idle footprint of each system in its default self-host configuration.

Table 2 — Autonomy quantified: services, RAM, cold start, mandatory keys, setup commands. Headline: **~100× less RAM idle than EverOS.** Competitor numbers are *estimates* derived from each project's docker-compose defaults and documented system requirements (sources cited in the row). The nox-mem row is [**estimated — not measured in prod for this revision**]; a fresh `ps -o rss=` measurement on the production VPS is queued as a §7.2 future-work item and the value will be tightened in the next paper revision (see footnote [18]).

System	Services	RAM idle	Cold start	Mandatory third-party keys	Setup commands	Sources
nox-mem	**1** (SQLite file + Node process)	**~50 MB** [estimated]	**<1 s**	**0** (offline-OK; embeddings optional)	**1** (`npm i && nox-mem reindex`)	This work; [19]
mem0	2 (Postgres + Qdrant)	~800 MB	~15 s	1 (OpenAI for embeddings)	~5	mem0 docker-compose defaults [20]

[17] Q/A/P strategic pivot of 2026-05-17 — three pillars (**Q**uality, **A**utonomy, **P**roduct). See `docs/ROADMAP.md` and `[[qap-pillars-strategic-decision]]` in the project memory.

[18] The `~50 MB` RAM figure for nox-mem in Table 2 is **estimated** (Node baseline ~30 MB + better-sqlite3 cache ~20 MB) and not measured on the production VPS for this revision. A `ps -o rss=,cmd= -ax | grep nox-mem` snapshot on the production host is queued as future work and will replace the estimate in the next paper revision. Permission to access production was denied during this revision; the estimate is conservative (upper-bound).

[19] The `~50 MB` RAM figure for nox-mem in Table 2 is **estimated** (Node baseline ~30 MB + better-sqlite3 cache ~20 MB) and not measured on the production VPS for this revision. A `ps -o rss=,cmd= -ax | grep nox-mem` snapshot on the production host is queued as future work and will replace the estimate in the next paper revision. Permission to access production was denied during this revision; the estimate is conservative (upper-bound).

[20] mem0 default self-host requires Postgres + Qdrant + an OpenAI key for embeddings (or a configured alternative provider). Counts: 2 services + 1 mandatory third-party key. Source: mem0 README and `docker-compose.yml` defaults at github.com/mem0ai/mem0.

System	Services	RAM idle	Cold start	Mandatory third-party keys	Setup commands	Sources
Letta	3 (Letta server + Postgres + OpenAI)	~1.5 GB	~30 s	1 (OpenAI)	~8	Letta self-host guide [21]
Zep OSS	2 (Zep + Postgres)	~1.2 GB	~30 s	**1 mandatory** (OpenAI for embeddings — hardcoded)	~6	Zep README [22]
EverOS / EverMind-AI	**5** (MongoDB + Elasticsearch + Milvus + Redis + Postgres)	**~4 GB+**	**~60 s**	2–3 (LLM + embedding + optional reranker)	~15+	EverMind-AI docker-compose [23]
LightRAG	2 (Neo4j + vector DB)	~1 GB	~20 s	1 (LLM provider for KG extraction)	~6	LightRAG repo defaults [24]

Reading the table. Three rows of the cost matrix translate directly into

[21] Letta default self-host requires the Letta server, Postgres, and an OpenAI key (or alternative LLM provider) for the agent loop. Counts: 3 components + 1 mandatory third-party key. Source: Letta self-host documentation at docs.letta.com and github.com/letta-ai/letta.

[22] Zep OSS requires the Zep service container + Postgres, and *hardcodes* OpenAI as the embedding provider in the default build (mandatory key, no fallback in OSS distribution). Counts: 2 services + 1 hardcoded mandatory key. Source: Zep README at github.com/getzep/zep.

[23] EverMind-AI / EverOS docker-compose declares MongoDB + Elasticsearch + Milvus + Redis + Postgres = 5 services, plus 2–3 third-party API keys for LLM, embedding, and (optional) reranker. Counts confirmed against the published `docker-compose.yml` in the EverMind-AI repo. The ~4 GB RAM-idle figure is the sum of documented minimum requirements for each service's container.

[24] LightRAG defaults to Neo4j for the knowledge graph + a vector store (Qdrant/Milvus/etc.), plus one LLM provider key for entity/relation extraction during indexing. Counts: 2 services + 1 mandatory third-party key. Source: LightRAG README at github.com/HKUDS/LightRAG.

Autonomy:

1. **Services column.** Every additional service is an additional failure mode, an additional security-patching surface, and an additional vendor that must be available on the day a user spins up the system. nox-mem ships as a single Node process operating on a single SQLite file; the only durable on-disk artifact is `nox-mem.db`. mem0/Zep/Letta/LightRAG each require >=1 database container and at least one external LLM/embedding provider. EverOS requires five containers, three of which are heavyweight infrastructure (MongoDB, Elasticsearch, Milvus). The single-service property is what makes "open `nox-mem.db` in `sqlite3` and inspect everything" a literal operation, not a euphemism.

2. **Mandatory third-party keys column.** A system that requires an OpenAI key by default is not autonomous regardless of license — the user is dependent on one specific vendor's pricing, rate limits, and terms of service. nox-mem treats embeddings as optional (FTS5-only retrieval is a valid degraded mode; §4) and is provider-agnostic when embeddings are enabled (Gemini default, Ollama-local feasible — §7.1 L2). Zep and Letta hardcode OpenAI as the default embedding provider.

3. **Cold start column.** A `<1s` cold start is what makes self-host *try-before-deciding* — the user can `npm i`, run one command, see results, and decide. A `~60s` cold start with five containers is what makes EverOS effectively a "build a small team to evaluate" decision, not an individual decision.

Caveat — RAM measurement methodology. The competitor RAM figures are *idle* (i.e., process started, no queries served, no ingestion in progress) and are *estimates* read from each project's documented system requirements and `docker stats` defaults in the published docker-compose files. They are not from a head-to-head benchmark on a single host. A side-by-side measurement on a controlled 4-vCPU / 8-GB host is a §7.2 future-work item (F-cost-bench). The nox-mem `~50 MB` figure should likewise be treated as an upper-bound estimate based on Node baseline + better-sqlite3 cache; a real `ps -o rss=` measurement on the production VPS is pending (see [25]) and will replace the estimate in the next revision.

---

[25] The `~50 MB` RAM figure for nox-mem in Table 2 is **estimated** (Node baseline ~30 MB + better-sqlite3 cache ~20 MB) and not measured on the production VPS for this revision. A `ps -o rss=,cmd= -ax | grep nox-mem` snapshot on the production host is queued as future work and will replace the estimate in the next paper revision. Permission to access production was denied during this revision; the estimate is conservative (upper-bound).

7. Limitations and Future Work

7.1 Limitations

L1 — Explicit-ingestion dependency (no zero-shot corpus coverage) nox-mem retrieves only what has been explicitly ingested via `ingestFile()`, `ingest-entity`, or the inotifywait watcher pipeline. There is no mechanism to answer queries over arbitrary external corpora at query time. This is a deliberate design constraint — the system is optimized for an agent's *own* accumulated memory, not general-purpose retrieval augmentation — but it means that coverage is bounded by ingestion discipline. A corpus that has never been ingested produces zero recall regardless of query quality. Users bootstrapping the system must explicitly run `nox-mem reindex` over existing files before the hybrid search layer is useful. Ref: `specs/2026-03-14-nox-memory-system-design.md`, ingestion pipeline §3.1.

L2 — Gemini API dependency for embeddings (cost + outbound network) The semantic retrieval layer (Layer 2) depends on Google's `gemini-embedding-001` model (3072 dimensions). This introduces two constraints: (a) every vectorization call requires outbound network access and a valid `GEMINI_API_KEY`, meaning an air-gapped deployment falls back to FTS5-only retrieval with no semantic recall; (b) API cost scales with corpus size — at the Sat 2026-05-24 corpus of ~69k chunks, a full re-vectorization pass takes approximately 30–40 minutes at quota limits of the free tier. The Autonomy pillar of the Q/A/P strategy explicitly calls out "provider your choice, zero vendor lock-in" as a long-term goal; local embedding substitution (e.g., `nomic-embed-text` via Ollama) is architecturally feasible but not validated against the canonical eval set. BYOK partial autonomy is available: users who supply their own Gemini API key operate without per-query billing exposure in the default free tier. Ref: `docs/DECISIONS.md` (model selection), `[[default-flash-lite-for-agent-infra-tasks]]`.

L3 — Single-instance architecture (no distributed sharding or replication) The system runs on a single SQLite file per agent database. WAL mode provides concurrent read safety, but there is no horizontal sharding, no replication across nodes, and no distributed coordination layer. The current production corpus (69k chunks across 7 databases on a 4-vCPU / 8GB KVM4) operates comfortably within these bounds, but the architecture does not generalize to multi-tenant deployments or corpora significantly exceeding the single-node memory/storage envelope. Distributed SQLite extensions (e.g., `cr-sqlite` CRDT-based replication) exist but are explicitly out of scope for v1. This is a known architectural decision, not an oversight. Ref: `docs/DECISIONS.md` (single-instance rationale).

L4 — No write-side concurrency control (last-writer-wins) Chunk ingestion operates under an optimistic concurrency model: `ingestFile()` deletes

existing chunks for the source file and re-inserts in a single transaction, but there is no row-level locking or version fence against concurrent ingest of the same source file from two processes. In practice the inotifywait watcher and manual CLI calls rarely overlap, and the WAL journal prevents data corruption; however, two concurrent ingest calls on the same file produce non-deterministic chunk counts. The `withOpAudit()` wrapper (`src/lib/op-audit.ts`) does not add a mutual-exclusion layer for ingest — it targets destructive bulk operations (reindex, consolidate, crystallize). Production mitigations are operational (systemd service prevents concurrent watcher processes; cron stagger of 5 minutes between agents), not architectural. Ref: `docs/INCIDENTS.md#2026-04-25`, `[[a1-op-audit-module]]`.

L5 — Evaluation sample size and canonical run gap (n=20 smoke vs n=100 target) The Sat 2026-05-24 cross-system comparison (§6.3) is based on a 20-query smoke over an eval-isolated DB (5,882 LoCoMo + 940 LongMemEval chunks), not the pre-registered canonical 100-query × 2-dataset × 6-system run. The canonical run was in progress at the time of this writing (5/6 competitor adapters under setup). All competitive figures for Mem0, Zep, Letta, agentmemory, and EverMind-AI in §6 carry `[PENDING canonical run]` tags and should not be treated as settled results. The nox-mem smoke figure (nDCG@10 = 0.6380 combined, p50 = 12 ms) is validated on the methodology but not directly comparable to the G5 V3 entity-eval figure (0.6237) because the eval corpus and query set differ. Ref: `specs/2026-05-23-Q4-comparison-execution-plan.md`, `[[q4-smoke-sat-2026-05-24-real-numbers]]`.

L6 — Cross-system comparison is methodologically partial Three of five competitors could not be evaluated against the full canonical corpus at the time of writing. Mem0 ran against a 500-chunk corpus cap imposed by cost-control constraints ($0.10 estimated ingest cost at full corpus), producing a nDCG@10 = 0.8569 on 20 queries that cannot be directly compared to nox-mem's full-corpus score — a smaller, more concentrated corpus tends to inflate nDCG for systems that retrieve all relevant documents. Zep and Letta require Docker compose setups that were in progress. EverMind-AI's repository was unavailable at time of access. The "concentration vs coverage trade-off" (high nDCG on capped corpus vs recall breadth on full corpus) is a genuine open question for per-system fair comparison, not a methodological failure. Ref: §6.3 anti-cherry-pick statement, `docs/COMPARISON.md`, `[[q4-partial-cross-system-sat-2026-05-24]]`.

L7 — Latency comparison conflates transport classes The Sat 2026-05-24 latency figures compare nox-mem (local FTS5 + sqlite-vec with Gemini API call for query embedding) against competitor adapters that go over HTTP or Python SDK to local Docker services. The nox-mem p50 = 12 ms figure reflects localhost UNIX-domain retrieval; the Mem0 p50 = 273 ms reflects a Docker-in-Docker HTTP call. These are not the same transport class. A valid latency

comparison requires a normalized transport — either all systems behind the same HTTP gateway, or all measured at the SDK level without network hop differences. The §6 latency figures are reported with this caveat in the per-system notes and should not be interpreted as head-to-head speed claims. Ref: `[[q3-latency-numbers-2026-05-18]]`, `docs/PERFORMANCE.md`.

L8 — Pain signal is directional but not statistically significant in isolation The "pain-weighted hybrid memory" framing rests primarily on the additive salience formula (§5.2) and section-aware ranking (§5.3). The `pain` dimension contributes W_PAIN = 0.10 of the salience weight, but its isolated causal contribution has not been validated to statistical significance: the E10 pain ablation (`paper/publication/paper-draft-sec4-7.md` §5.5) reports $\Delta = +0.0065$ with 95% CI $[-0.0143, +0.0338]$ on n = 31, directional but not significant. The current production corpus has 90.67% of chunks at the default `pain = 0.2`, providing insufficient variance for a precise estimate. A definitive pain signal ablation requires a corpus where pain scores span the full [0.1, 1.0] range. Ref: §5.7 honest characterization, `[[d47-path-c-decision]]`.

7.2 Future Work

F1 — A2 Tier 3 P5: production-ready encrypted memory (in flight) Phase 5 of the A2 Tier 3 roadmap targets a full SQLCipher-encrypted memory store with Ed25519-signed audit checkpoints (P4 deployed via PR #294). The signed checkpoint chain enables tamper-evident audit across destructive operations (`reindex`, `consolidate`, `crystallize`) without requiring a central trust authority. P5 closes the Tier 3 arc by integrating encryption key management with the existing `withOpAudit()` wrapper and the Tier 3 reads-audit layer (P3, PR #292/#293). Target deployment: post-GTM Phase 2 launch, estimated Sun 2026-05-25. Ref: `specs/2026-05-24-A2-tier3-crypto-audit-RECON.md`, `docs/A2-TIER3-MIGRATION-RUNBOOK.md`, `[[a1-op-audit-module]]`.

F2 — F10 Phase C/D: shadow tracker empirical A/B for ranking changes F10 Phase A (`/observability/health.html`) and Phase B (`/observability/evals.html`) are deployed (§5.6, decision D53). Phase C targets a shadow-mode query logger that captures production queries, executes them against a candidate ranking config in parallel, and accumulates query-level nDCG deltas before any promotion decision. Phase D operationalizes this into a pre-promotion gate: any ranking change (boost weight adjustment, mutex threshold, salience weight) that has not accumulated >=50 shadow queries with $p < 0.05$ improvement is blocked from reaching the production endpoint. This closes the observability gap identified in `[[ship-ranking-changes-in-shadow-mode-first]]` — currently the shadow mode is a flag toggle, not

an integrated eval pipeline. Ref: `docs/ROADMAP.md` F10, decision D53, PR #207/#212.

F3 — Per-method benchmark Phase B: cross-method nDCG optimization The Q4 per-method benchmark (§6, `specs/2026-05-21-per-method-benchmark-comparison.md`) establishes the cross-system baseline. Phase B targets per-query-type boost calibration: given that keyword queries respond differently from natural-language queries (G10c §5.5), and single-hop vs multi-hop have opposing mutex trade-offs (G10b §5.5), a routing layer that selects ranking parameters based on query-type classification has measurable potential upside. Estimated Lab Q1 item. Ref: `specs/2026-05-21-per-method-benchmark-comparison.md`, `[[g10c-per-style-mutex-2026-05-21]]`.

F4 — EverMemBench equivalence: honest comparison against EverMind-AI dataset `[[everos-honest-comparison-benchmark-gap]]` identifies that EverMind-AI publishes standardized results on EverMemBench (EverCore 83% LongMemEval / 93% LoCoMo; HyperMem 92.73% LongMemEval). Running nox-mem on EverMemBench with the same evaluation protocol closes the benchmark gap and provides a reviewer-grade comparison for the arXiv submission. This is gated on EverMind-AI repository availability (currently unavailable) or an alternative comparable dataset. Estimated Lab Q1 priority if the repository returns. Ref: `[[everos-benchmark-publisher-competitor]]`, `[[benchmark-gap-longmemeval-locomo]]`.

F5 — Neural reranker: cross-encoder rerank post-RRF The current retrieval stack terminates at RRF fusion (§4.1). A cross-encoder reranker — receiving the top-K RRF candidates and the original query as a pair — is the standard next step in multi-stage retrieval and typically yields +3–8% nDCG@10 over bi-encoder baselines (see e.g., Nogueira & Cho 2019 on MS MARCO). The Autonomy constraint (`[[neural-reranker-evolution-vector]]`) favors a locally-runnable cross-encoder (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2` via sentence-transformers, ~66MB) over a cloud inference call, keeping the retrieval stack fully offline-capable. Estimated Lab Q1/Q2. Ref: `[[neural-reranker-as-vetor-evolutivo-pos-rrf]]`, `docs/ROADMAP.md` Lab Q1.

F6 — Lab Q1 scale validation: 250k chunk corpus The current production corpus is 69,495 chunks (as of G10, 2026-05-20). The Lab Q1 roadmap targets a 250k chunk corpus to validate: (a) sqlite-vec ANN recall at scale (current exact-search; approximate search becomes necessary past ~100k vectors at reasonable latency targets); (b) salience formula stability (the recency component decays over a longer history window); (c) FTS5 BM25 IDF calibration (with more documents, rare-term IDF weights shift). The `[[lab-q1-scale-250k]]` item has no committed spec yet; it is gated on `NOX_SALIENCE_MODE=active` remaining stable through the GTM Phase

2 feedback cycle. Ref: `docs/ROADMAP.md` Lab Q1, `[[q-a-p-pillars-strategic-pivot-2026-05-17]]`.

F7 — Multilingual corpus coverage: Portuguese and Spanish The current evaluation corpus is English-dominant (LongMemEval and LoCoMo are English datasets; the internal entity-eval golden set mixes English and Portuguese). The FTS5 `unicode61` tokenizer with porter stemmer does not stem Portuguese or Spanish tokens correctly (e.g., "decisão" stems to "decisa" rather than "decid-"; Spanish gerunds lose morphological overlap). The Gemini semantic layer partially compensates via cross-lingual embedding space, but there is no explicit multilingual evaluation. A Portuguese golden set is a natural next step given the production operational language of the corpus. This is deferred to post-launch community feedback intake.

F8 — GTM Phase 2 launch and community feedback intake The Q/A/P roadmap (decision `[[qap-pillars-strategic-pivot-2026-05-17]]`) defines GTM Phase 2 as gated on D43 (nox-mem top-3 in >=2 of 4 key metrics — nDCG@10, R@10, MRR, latency). The target launch date is Wed 2026-06-03, conditional on the canonical Q4 run completing and D43 passing. Post-launch, community feedback from the OSS release is expected to surface real-world limitation patterns not visible in the synthetic golden sets (e.g., corpora with high image-to-text OCR content, multi-language mixes, or very short memory fragments < 20 words that the current chunker merges). The feedback cycle directly informs Lab Q2 priorities. Ref: `docs/ROADMAP.md` GTM Phase 2, `docs/gtm/`, `[[overnight-automode-push-pattern]]`.

---

8. Knowledge Graph v2

8.1 Entity Extraction

v1 (Regex-based): Used hardcoded regular expressions for 3 entity types (person, project, agent) with a static alias map for name normalization. Limited to predefined names, producing 26 entities.

v2 (LLM-powered): Uses Ollama llama3.2:3b with a structured extraction prompt. Each chunk is processed with temperature 0.1 for deterministic output. The LLM returns JSON with entities (name + type) and relations (source + relation + target).

Extraction results after processing 866 chunks:

Metric	Regex v1	LLM v2	Improvement
Entities	26	384	14.8x
Relations	59	529	9.0x
Entity Types	3	11	3.7x

Metric	Regex v1	LLM v2	Improvement

Entity Type Distribution:

Type	Count	Description
project	109	Software projects, products, repos
tool	67	Libraries, frameworks, CLI tools
concept	54	Abstract ideas, patterns, methodologies
person	53	Team members, contacts, stakeholders
organization	50	Companies, teams, departments
agent	45	AI agents in the fleet
location	2	Geographic references
other	4	Device, currency, date, computer

8.2 Temporal Decay and TTL

Relations have a 90-day time-to-live (TTL) from creation. The confidence decay mechanism operates as follows:

1. Relations start with confidence 0.8 (extracted) or 0.9 (confirmed)
2. Every 30 days without re-confirmation, confidence drops by 0.1
3. Relations below 0.3 confidence receive accelerated 7-day expiry
4. Expired relations are deleted during `kg-prune` execution
5. Re-confirmation (observing the same relation in new chunks) resets confidence to 0.9 and extends TTL by 90 days

This mechanism ensures the knowledge graph naturally forgets stale information while reinforcing actively observed patterns.

8.3 Decision Versioning

Architectural decisions are tracked with full version history in the `decision_versions` table. Each decision has a unique key (e.g., `dedup-strategy`, `fallback-chain`) and supports:

- Version chains with supersession tracking
- Authorship attribution
- Source file provenance
- Current vs. historical querying

10 decisions are currently tracked, covering API key management, LLM fallback chains, embedding model selection, agent isolation strategy, and synchronization schedules.

8.4 Graph Traversal

The `findPath()` function implements BFS (Breadth-First Search) to discover shortest paths between any two entities. This enables queries like "How is Toto connected to nox-mem?" which traverses person → project → tool → agent relationships. Maximum depth is configurable (default: 4 hops).

---

9. Cross-Agent Intelligence

9.1 Agent Expertise Profiling

Each agent's memory is analyzed to determine its unique expertise based on chunk type distribution. The dominant chunk type determines the agent's strength category:

- **daily** → "Daily operations & activity logging"
- **team** → "Team coordination & shared knowledge"
- **decision** → "Decision tracking & rationale"
- **lesson** → "Lessons learned & pattern recognition"

Profiles include chunk counts, type breakdowns, top topics (via FTS5 term frequency), and last activity dates.

9.2 Knowledge Sharing

The `pullInsightsFrom()` function enables any agent to query lessons and decisions from other agents without direct database access. This creates a knowledge transfer mechanism where, for example, Cipher (Security) can learn from Forge's (Code Reviewer) past code review decisions.

`pullAllInsights()` aggregates insights across all agents, sorted by date, providing a fleet-wide learning feed.

9.3 Cross-Agent Knowledge Graph Merge

`mergeCrossKnowledgeGraphs()` scans all agent databases for kg_entities and kg_relations tables, merging them into a unified entity view. Entities are matched by type + lowercase name. The output shows which entities are known to which agents and their combined mention counts, enabling identification of shared knowledge vs. agent-specific expertise.

---

10. MCP Server Interface

nox-mem exposes 14 tools via the Model Context Protocol (MCP) over stdio (JSON-RPC 2.0):

Tool	Category	Description
nox_mem_search	Retrieval	Hybrid search (FTS5 + semantic + RRF)
nox_mem_stats	Monitoring	Database statistics and health
nox_mem_primer	Context	Session recovery summary (~500 tokens)
nox_mem_ingest	Ingestion	Index a file into memory
nox_mem_cross_search	Cross-Agent	Search across all 7 databases
nox_mem_cross_stats	Cross-Agent	Chunk counts per agent
nox_mem_metrics	Monitoring	Daily observability metrics
nox_mem_kg_build	KG	Build knowledge graph from chunks
nox_mem_kg_query	KG	Query entity and its relations
nox_mem_kg_stats	KG	Knowledge graph statistics
nox_mem_agent_profiles	Intelligence	Agent expertise profiles
nox_mem_cross_kg	Intelligence	Merged cross-agent knowledge graph
nox_mem_kg_path	Intelligence	BFS path between entities
nox_mem_self_analysis	Analysis	Contradiction detection, pattern analysis

---

11. HTTP API Server

A lightweight HTTP API (Node.js built-in `http` module, zero dependencies) runs on port 18800, exposing memory data to the React dashboard:

Endpoint	Method	Response
`/api/health`	GET	System health: chunks, consolidation, vector coverage, services, KG stats, DB size
`/api/agents`	GET	Agent expertise profiles array
`/api/kg`	GET	Knowledge graph entities and relations
`/api/kg/path?from=X&to=Y`	GET	BFS shortest path between entities
`/api/search?q=QUERY&limit=N`	GET	Hybrid search results
`/api/cross-kg`	GET	Merged cross-agent knowledge graph

CORS headers are set for cross-origin access from the Vercel-hosted dashboard.

12. Operational Infrastructure

12.1 Cron Schedule

24 cron jobs manage automated operations:

Time	Frequency	Job	Details
23:00-23:25	Daily	Agent consolidation	6 agents, 5-min stagger, reindex→consolidate
23:30	Daily	Workspace consolidation	Central workspace daily notes
23:35	Daily	Session wrap-up	SESSION-STATE.md, Notion sync, git commit
04:00	Weekly (Sun)	Vectorize	Gemini embeddings for new/changed chunks
*/5 min	Continuous	Health check	Watcher heartbeat, service liveness
02:00	Daily	SQLite backup	Online backup API, 7-day retention pruning
*/6 hours	Continuous	Git backup	Auto-commit memory file changes
09:00	Weekly (Mon)	Token check	Forge CC token verification

12.2 Backup Strategy

Three backup mechanisms operate independently:

1. **SQLite Online Backup**: Uses better-sqlite3's backup API for crash-consistent copies. Daily at 02:00, 7-day retention with automatic pruning.
2. **Git Auto-Commit**: Memory directory changes are committed every 6 hours, providing full change history.
3. **File System**: WAL mode ensures database consistency during concurrent reads/writes.

12.3 LLM Fallback Chain

To ensure continuous operation regardless of provider availability:

Paid Tier: Claude Opus → Sonnet → Haiku → GPT-5.1 → Gemini 2.5
Free Tier: Nemotron → Groq Llama70B → Healer → Hunter → Trinity → Gemma27B

The fallback is configured in the environment and selected at runtime based on task complexity and availability.

---

13. Dashboard Integration

The TotoClaw Command Center (React 18 + TypeScript + Vite + shadcn/ui) provides 11 pages including 4 nox-mem-specific views:

- **Memory Health** (`/memory`): Real-time system stats, vector coverage progress bar, service status indicators, agent breakdown table
- **Knowledge Graph** (`/knowledge-graph`): Interactive force-directed canvas graph, entity type filters, BFS path finder
- **Agent Intel** (`/agent-intel`): Agent expertise cards with type distribution bars, hybrid search interface, cross-agent knowledge entities
- **System Paper** (`/system-paper`): Live technical analysis with Recharts visualizations (pie, bar, radar, area charts), auto-refresh every 60 seconds

All data is fetched from the nox-mem API server via TanStack React Query with configurable polling intervals.

---

14. Evolution History

Version	Date	Key Changes
v1.0	Mar 14	SQLite FTS5, basic search, consolidation, Notion sync
v2.0	Mar 17	MCP server, systemd services, watcher heartbeat, primer
v2.2	Mar 20	Cross-agent search, KG v1 (regex), self-improve, decision versioning
v2.5	Mar 22	Multi-agent workspace fix (OPENCLAW_WORKSPACE), gateway supervision
v2.6	Mar 22	Hybrid search default (FTS5+Gemini+RRF), 866/866 vectorized
v3.0	Mar 23	KG v2 (LLM, 384 entities), Cross-Agent Intelligence, HTTP API, dashboard

Version	Date	Key Changes
v3.7	Apr 23	Schema V10 (`retention_days` v8 + `pain` v9 + `section` v10), entity file format, section_boost
Wave A	May 19	Additive salience formula, `tier_boost` off-by-default, `source_type` backfill (67,949 chunks), G5 V3 ablation (PRs #150 / #151 / #153)
G10 Hard Mutex	May 20	`section` ↔ `source_type` mutex deployed against `g9.db` 69,495 chunks (PRs #181 / #182)
G10d ACTIVE-T2	May 21	Conditional mutex gated by `query_entity_count ≤ 2`, deployed via systemd drop-in `NOX_MUTEX_QUERY_ENTITY_THRESHOLD=2` (PR #198, decision D51); multi-hop +1.58% nDCG / adversarial +3.04% nDCG recovered
F10 Phase A + B	May 21	Foundation observability dashboards (`/observability/health.html` + `/observability/evals.html`) deployed via Tailscale tunnel (PRs #207 / #212, decision D53)

15. Conclusion

nox-mem demonstrates that persistent, searchable, and shareable memory for AI agent fleets is achievable with commodity infrastructure (single VPS, SQLite, local LLM). The hybrid search system consistently outperforms single-method retrieval, particularly for multilingual content and compound technical terms. The LLM-powered knowledge graph provides 15x richer entity extraction compared to regex approaches, while temporal decay ensures the graph stays current without manual curation. The Wave A empirical evaluation (§5) cravou nDCG@10 = 0.6237 on the entity-flavored golden set (+78.8% relative over the G3 baseline), with `section_boost` identified as the dominant driver (99.85% of the lift recovered by A3 alone) and the additive salience formula validated by the `active > shadow` reversal. The G10d conditional mutex evolution (§5.5, deployed 2026-05-21) consolida o canonical boost stack `section_boost × source_type_boost (Hard Mutex gated by query_entity_count ≤ 2) × salience v2 additive` em produção, recu-

perando regressões multi-hop e adversarial com diluição contida em single-hop. A camada F10 (§5.6, decisão D53) torna o estado de produção verificável a qualquer momento via dashboards Phase A (`/observability/health.html`) + Phase B (`/observability/evals.html`).

A §6 Q4 COMPARISON está pre-registered (`specs/2026-05-23-Q4-comparison-execution-plan.md`) e o **smoke de Sat 2026-05-24 15h30 BRT** populou a primeira linha de §6.3 com números de nox-mem (nDCG@10=0.6380 combined, p50=12ms, gold-hit 13/20 em 20 queries dry-run-sample sobre eval-isolated DB de 5.882 LoCoMo + 940 LongMemEval chunks). O **run canônico** — 100 queries × 2 datasets × 6 sistemas (Mem0, Zep, Letta, agentmemory, EverMind-AI + nox-mem) — ainda está em execução com 5/6 competitor adapters em setup, e atualiza as células `[PENDING canonical run]` quando crava. O gate D43 (top-3 em >=2 das 4 métricas chave) é avaliado contra o run canônico; o smoke valida a metodologia + confirma que nox-mem retrieval funciona end-to-end, destravando a defesa pre-launch da GTM Phase 2.

The cross-agent intelligence layer transforms isolated agent memories into a collaborative knowledge base, enabling institutional learning across the fleet. Combined with the live dashboard, the system provides full observability into the collective memory of the agent organization.

Repository: github.com/totobusnello/nox-workspace **Dashboard:** github.com/totobusnello/agent-hub-dashboard **Spec:** Projetos/memoria-nox/specs/2026-03-14-nox-memory-system-design.md

---

References and Footnotes

Related-systems references

Internal references

Autonomy table (Table 2) sources